SQUIRREL MADE PRODUCTS

Square Checkout API

Developer Implementation Guide

Migrating from Square Payment Links to a Full Shopping Cart Experience

Version 1.0 · Prepared for Development Handoff · 2026
squirrelmadeproducts.com

1. Project Overview

This guide details the complete migration from Square Payment Links (single-item checkout) to the Square Checkout API with a multi-item shopping cart. The goal is to allow customers to add multiple products to a cart and check out in a single transaction.

1.1 The Problem

The current implementation uses Square-generated payment links assigned per product. These links open a Square-hosted checkout page for that one item only — there is no mechanism for a customer to add more than one product per transaction. This results in a poor user experience and likely reduces average order value.

1.2 The Solution

Replace individual payment links with:

- A client-side shopping cart built in JavaScript on the existing site
- A lightweight backend endpoint (serverless function recommended) that calls the Square API
- Square's hosted checkout page, which handles PCI-compliant payment collection

Square handles all payment processing. The developer only needs to build the cart UI and the API call.

1.3 What Changes vs. What Stays the Same

Component	Current State	After Migration
Payment processing	Square (via payment links)	Square (via Checkout API) — no change
Checkout UI	Square-hosted (per link)	Square-hosted (same experience)
Cart experience	None — one item only	Multi-item cart built on your site
Product pages / site	Unchanged	Add to Cart buttons replace Buy Now links
Inventory management	Square Dashboard	Square Dashboard — no change
Order confirmation	Square email	Square email — no change

2. Architecture Overview

The new system has three layers: the frontend cart, a backend API endpoint, and Square's hosted checkout.

2.1 Request Flow

1. Customer browses products on squirrelmadeproducts.com

2. Customer clicks Add to Cart — item is added to a JavaScript cart object stored in memory/sessionStorage
3. Customer views cart, adjusts quantities, and clicks Checkout
4. Frontend sends a POST request to your backend endpoint with the cart contents (array of line items)
5. Backend validates the request, then calls Square's POST /v2/online-checkout/payment-links endpoint with your Square credentials
6. Square returns a checkout_url
7. Backend returns the checkout_url to the frontend
8. Frontend redirects the customer to Square's hosted checkout page
9. Customer completes payment on Square's page
10. Square redirects customer to your confirmation/success URL
11. Square sends a webhook notification to your backend confirming the completed order

2.2 Architecture Diagram

```
[Customer Browser]
      |
      |  Add to Cart (JS)
      |
[Cart State]  ⟷  sessionStorage
      |
      |  POST /api/checkout  (cart JSON)
      ↓
[Your Backend]  (Vercel / Netlify / Node server)
      |
      |  POST /v2/online-checkout/payment-links
      |  Authorization: Bearer {SQUARE_ACCESS_TOKEN}
      ↓
[Square API]
      |
      |  Returns { payment_link: { url: '...' } }
      ↓
[Your Backend]  →  returns { checkoutUrl } to frontend
      ↓
[Customer Browser]  →  redirect to Square checkout
      ↓
[Square Hosted Checkout]
      |
      |  Payment complete → redirect to your success URL
      |  Webhook → POST /api/webhooks/square
```

2.3 Backend Recommendation

A serverless function is the simplest and most cost-effective backend for this use case. Recommended platforms:

Platform	Notes	Cost
Vercel Functions	Best if site is already on Vercel. Zero config.	Free tier generous
Netlify Functions	Best if site is on Netlify. Drop-in Lambda functions.	Free tier generous
AWS Lambda	More control, slightly more setup. Good for Node.js.	Free tier available

Platform	Notes	Cost
Express.js on VPS	If a traditional server is preferred. Any Node host.	Depends on host

IMPORTANT: Never call the Square API directly from the browser/frontend. Your Square Access Token must never be exposed in client-side code.

3. Square Developer Setup

3.1 Access the Square Developer Dashboard

12. Go to https://developer.squareup.com and sign in with the Squirrel Made Square account
13. Navigate to Applications → Create Application (or select existing)
14. Name it something like "Squirrel Made Checkout"

3.2 Credentials You Need

From the Square Developer Dashboard, collect the following:

Credential	Where to Find It	Used For
Access Token	App → Credentials tab → Production Access Token	Authenticating API requests
Location ID	App → Locations tab → select your location → copy ID	Required in checkout payload
Sandbox Access Token	App → Credentials tab → Sandbox Access Token	Testing before going live
Sandbox Location ID	Same tab — Sandbox section	Testing only

NOTE: Start all development and testing with Sandbox credentials. Switch to Production credentials only when ready to go live.

3.3 Environment Variables

Store all credentials in environment variables — never hardcode them in source files. Create a .env file locally (add to .gitignore) and set these on your hosting platform:

```
# .env (local development — NEVER commit this file)
SQUARE_ACCESS_TOKEN=your_sandbox_access_token_here
SQUARE_LOCATION_ID=your_sandbox_location_id_here
SQUARE_ENVIRONMENT=sandbox

# Production values (set in Vercel / Netlify dashboard)
SQUARE_ACCESS_TOKEN=your_production_access_token_here
SQUARE_LOCATION_ID=your_production_location_id_here
SQUARE_ENVIRONMENT=production
```

4. Backend Implementation

4.1 Install the Square Node.js SDK

Square provides an official Node.js SDK. Install it in your backend project:

```
npm install squareup
```

4.2 The Checkout Endpoint

Create a single POST endpoint at /api/checkout. This is the only endpoint your frontend needs to call. Below is a complete implementation:

```
// api/checkout.js  (Vercel serverless function)
// Works equally as a Netlify function or Express route

import { Client, Environment } from 'squareup';
import { randomUUID } from 'crypto';

const client = new Client({
  accessToken: process.env.SQUARE_ACCESS_TOKEN,
  environment: process.env.SQUARE_ENVIRONMENT === 'production'
    ? Environment.Production
    : Environment.Sandbox
});

export default async function handler(req, res) {
  if (req.method !== 'POST') {
    return res.status(405).json({ error: 'Method not allowed' });
  }

  const { lineItems } = req.body;

  // Basic validation
  if (!lineItems || !Array.isArray(lineItems) || lineItems.length === 0) {
    return res.status(400).json({ error: 'Cart is empty or invalid' });
  }

  // Map frontend cart items to Square's expected format
  const squareLineItems = lineItems.map(item => ({
    name: item.name,
    quantity: String(item.quantity),  // Square requires string
    basePriceMoney: {
      amount: BigInt(Math.round(item.price * 100)),  // Convert to cents
      currency: 'USD'
    }
  }));

  try {
    const response = await client.checkoutApi.createPaymentLink({
      idempotencyKey: randomUUID(),
      order: {
        locationId: process.env.SQUARE_LOCATION_ID,
        lineItems: squareLineItems
      },
```

```
      checkoutOptions: {
        redirectUrl: 'https://squirrelmadeproducts.com/order-confirmed',
        askForShippingAddress: true
      }
    });

    const checkoutUrl = response.result.paymentLink.url;
    return res.status(200).json({ checkoutUrl });

  } catch (error) {
    console.error('Square API error:', error);
    return res.status(500).json({ error: 'Failed to create checkout session' });
  }
}
```

4.3 Line Item Format Reference

Each item passed from the frontend should conform to this structure:

```
// Frontend cart item shape (what the frontend sends)
{
  name: 'Basil Infused EVOO',     // string — product name
  quantity: 2,                      // integer — number of units
  price: 18.99                      // float — price in USD (dollars, not cents)
}
```

Important notes on the line item format:

- quantity must be sent to Square as a string (the SDK/API requires this — convert with String(n))
- price must be converted to cents as a BigInt: BigInt(Math.round(price * 100))
- currency should always be 'USD' for this store
- name should match exactly what you want to appear on the Square receipt and order confirmation

4.4 Complete Request / Response Example

Request from Frontend → Backend

```
POST /api/checkout
Content-Type: application/json

{
  "lineItems": [
    { "name": "Basil Infused EVOO", "quantity": 2, "price": 18.99 },
    { "name": "Traditional Dark Balsamic", "quantity": 1, "price": 16.99 },
    { "name": "Herb & Garlic Spice Blend", "quantity": 1, "price": 12.99 }
  ]
}
```

Response from Square API

```
{
  "payment_link": {
    "id": "BQNLJWAJCTCNB",
    "version": 1,
    "order_id": "ORDER_ID_HERE",
```

```
    "url": "https://checkout.square.site/pay/BQNLJWAJCTCNB",
    "created_at": "2026-01-15T12:00:00Z"
  },
  "related_resources": { ... }
}
```

Response from Your Backend → Frontend

```
{
  "checkoutUrl": "https://checkout.square.site/pay/BQNLJWAJCTCNB"
}
```

---

5. Frontend Implementation

5.1 Cart State

The cart is a JavaScript array of line item objects stored in sessionStorage. It persists across page navigations within the same browser session, and is cleared when the session ends.

```
// cart.js — Cart state management module

const CART_KEY = 'squirrel_made_cart';

export function getCart() {
  const stored = sessionStorage.getItem(CART_KEY);
  return stored ? JSON.parse(stored) : [];
}

export function saveCart(cart) {
  sessionStorage.setItem(CART_KEY, JSON.stringify(cart));
}

export function addToCart(product) {
  const cart = getCart();
  const existing = cart.find(item => item.name === product.name);
  if (existing) {
    existing.quantity += 1;
  } else {
    cart.push({ name: product.name, price: product.price, quantity: 1 });
  }
  saveCart(cart);
  updateCartUI();
}

export function removeFromCart(productName) {
  const cart = getCart().filter(item => item.name !== productName);
  saveCart(cart);
  updateCartUI();
}

export function updateQuantity(productName, quantity) {
  const cart = getCart();
  const item = cart.find(i => i.name === productName);
  if (item) {
    if (quantity <= 0) {
```

```
      removeFromCart(productName);
    } else {
      item.quantity = quantity;
      saveCart(cart);
      updateCartUI();
    }
  }
}

export function getCartTotal() {
  return getCart().reduce((sum, item) => sum + (item.price * item.quantity), 0);
}

export function getCartCount() {
  return getCart().reduce((sum, item) => sum + item.quantity, 0);
}

export function clearCart() {
  sessionStorage.removeItem(CART_KEY);
  updateCartUI();
}

function updateCartUI() {
  // Update cart count badge in header
  const badge = document.getElementById('cart-count');
  if (badge) badge.textContent = getCartCount();
}
```

5.2 Add to Cart Buttons

Replace the existing Buy Now / payment link buttons on product pages with Add to Cart buttons. Each button needs the product name and price as data attributes:

```
<!-- Product card HTML — replace Buy Now link with this -->
<button
  class="add-to-cart-btn"
  data-name="Basil Infused EVOO"
  data-price="18.99"
>
  Add to Cart
</button>

<!-- Vanilla JS event binding (add to your main JS file) -->
document.querySelectorAll('.add-to-cart-btn').forEach(btn => {
  btn.addEventListener('click', () => {
    addToCart({
      name: btn.dataset.name,
      price: parseFloat(btn.dataset.price)
    });
    // Optional: Show a brief confirmation animation
    btn.textContent = 'Added!';
    setTimeout(() => btn.textContent = 'Add to Cart', 1500);
  });
});
```

5.3 Checkout Flow

When the customer clicks Checkout from the cart, call your backend and redirect to Square:

```
// checkout.js
```

```
async function initiateCheckout() {
  const cart = getCart();

  if (cart.length === 0) {
    alert('Your cart is empty.');
    return;
  }

  // Show loading state
  const checkoutBtn = document.getElementById('checkout-btn');
  checkoutBtn.textContent = 'Loading...';
  checkoutBtn.disabled = true;

  try {
    const response = await fetch('/api/checkout', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ lineItems: cart })
    });

    if (!response.ok) throw new Error('Checkout request failed');

    const { checkoutUrl } = await response.json();

    // Redirect to Square's hosted checkout
    window.location.href = checkoutUrl;

  } catch (error) {
    console.error('Checkout error:', error);
    alert('Something went wrong. Please try again.');
    checkoutBtn.textContent = 'Checkout';
    checkoutBtn.disabled = false;
  }
}
```

5.4 Cart UI Component

Here is a minimal cart drawer/panel structure. Style to match the Squirrel Made brand:

```
<!-- Cart panel — place in your site's HTML, hidden by default -->
<div id="cart-panel" class="cart-panel hidden">
  <div class="cart-header">
    <h2>Your Cart</h2>
    <button id="close-cart">×</button>
  </div>
  <div id="cart-items"><!-- Dynamically rendered --></div>
  <div class="cart-footer">
    <div class="cart-total">
      Total: <span id="cart-total">$0.00</span>
    </div>
    <button id="checkout-btn" onclick="initiateCheckout()">
      Checkout
    </button>
  </div>
</div>

<!-- Cart icon in header (shows item count) -->
<button id="open-cart">
  🛒 Cart (<span id="cart-count">0</span>)
</button>
```

Cart render function:

```
function renderCart() {
  const cart = getCart();
  const container = document.getElementById('cart-items');
  const totalEl = document.getElementById('cart-total');

  if (cart.length === 0) {
    container.innerHTML = '<p>Your cart is empty.</p>';
    totalEl.textContent = '$0.00';
    return;
  }

  container.innerHTML = cart.map(item => `
    <div class="cart-item">
      <span class="item-name">${item.name}</span>
      <span class="item-price">$${item.price.toFixed(2)}</span>
      <input
        type="number"
        value="${item.quantity}"
        min="0"
        onchange="updateQuantity('${item.name}', parseInt(this.value))"
      />
      <button onclick="removeFromCart('${item.name}')">Remove</button>
    </div>
  `).join('');

  totalEl.textContent = '$' + getCartTotal().toFixed(2);
}
```

6. Webhooks (Order Confirmation)

Webhooks allow Square to notify your server when a payment is completed. This is how you know an order was placed — and can trigger any post-purchase logic (confirmation emails, inventory updates, order logging, etc.).

6.1 Setting Up Webhooks in Square

15. In the Square Developer Dashboard, go to Webhooks
16. Click Add Webhook
17. Set the endpoint URL to: https://squirrelmadeproducts.com/api/webhooks/square
18. Select the event: payment.completed (and optionally order.fulfillment.updated)
19. Copy the Webhook Signature Key — you will need this to verify incoming requests

6.2 Webhook Handler

```
// api/webhooks/square.js

import { WebhooksHelper } from 'squareup';

export default async function handler(req, res) {
```

```
  if (req.method !== 'POST') {
    return res.status(405).end();
  }

  const signature = req.headers['x-square-hmacsha256-signature'];
  const body = JSON.stringify(req.body);
  const signatureKey = process.env.SQUARE_WEBHOOK_SIGNATURE_KEY;
  const notificationUrl = 'https://squirrelmadeproducts.com/api/webhooks/square';

  // Verify the webhook is genuinely from Square
  const isValid = WebhooksHelper.isValidWebhookEventSignature(
    body,
    signature,
    signatureKey,
    notificationUrl
  );

  if (!isValid) {
    console.warn('Invalid webhook signature — rejected');
    return res.status(401).end();
  }

  const event = req.body;

  if (event.type === 'payment.completed') {
    const payment = event.data.object.payment;
    console.log('Payment completed:', payment.id, '— Amount:', payment.totalMoney);
    // Add any post-payment logic here:
    // - Send internal notification
    // - Log to database
    // - Trigger fulfillment workflow
  }

  // Always return 200 quickly — Square will retry if you don't
  return res.status(200).end();
}
```

IMPORTANT: Always return HTTP 200 from your webhook handler as quickly as possible. Square will retry delivery if it doesn't receive a 200 within a few seconds. Run any slow processing asynchronously.

7. Order Confirmed Page

After a successful payment, Square redirects the customer to the URL you specified in checkoutOptions.redirectUrl. Create a simple confirmation page at that URL.

```
// In your backend checkout handler, set this:
checkoutOptions: {
  redirectUrl: 'https://squirrelmadeproducts.com/order-confirmed',
  askForShippingAddress: true
}

// On the /order-confirmed page, clear the cart:
// (place in the page's JS)
clearCart();
```

The confirmation page should:

- Thank the customer for their order
- Let them know they will receive a confirmation email from Square
- Call clearCart() to empty the cart state
- Provide a link back to the shop to encourage repeat purchases

8. Testing

8.1 Sandbox Testing

Square provides a full sandbox environment with test cards. All development should be done against sandbox before switching to production.

Test Card Number	CVV	Expiry	Result
4111 1111 1111 1111	Any 3 digits	Any future date	Successful payment
4000 0000 0000 0002	Any 3 digits	Any future date	Card declined
4000 0000 0000 0069	Any 3 digits	Any future date	Expired card
4000 0000 0000 0127	Any 3 digits	Any future date	Incorrect CVV

8.2 Testing Checklist

Before going live, verify each of the following:

- Add a single item to cart — quantity increments correctly
- Add multiple different items — all appear in cart
- Add the same item twice — quantity updates rather than duplicating
- Remove an item — cart updates correctly
- Update quantity to 0 — item is removed from cart
- Cart count badge in header reflects total item count
- Cart total displays the correct dollar amount
- Click Checkout — loading state shows on button
- Sandbox checkout opens — correct items and prices shown in Square
- Complete payment with test card — redirected to /order-confirmed
- Cart is cleared after redirect to /order-confirmed
- Webhook fires and logs correctly after test payment
- Declined card test — Square shows error, customer can retry
- Empty cart checkout — blocked with appropriate message
- Test on mobile viewport — cart UI usable on small screens

8.3 Switching to Production

When testing is complete and you are ready to go live:

20. In your hosting platform (Vercel/Netlify/etc.), update environment variables:

```
SQUARE_ACCESS_TOKEN=your_PRODUCTION_access_token
SQUARE_LOCATION_ID=your_PRODUCTION_location_id
SQUARE_ENVIRONMENT=production
```

21. Update the webhook endpoint URL to your production domain in Square Dashboard
22. Update the SQUARE_WEBHOOK_SIGNATURE_KEY to the production webhook signature key
23. Do one final test with a real card for a small amount (e.g., $1.00 test product) to confirm end-to-end flow
24. Remove any test/debug console.log statements
25. Remove all Buy Now payment link buttons from product pages

---

9. Security Checklist

Item	Requirement	Status
API Keys	NEVER in frontend code. Environment variables only.	[ ] Verify
.gitignore	.env file listed in .gitignore before first commit.	[ ] Verify
HTTPS	All endpoints must be served over HTTPS in production.	[ ] Verify
Input validation	Validate lineItems array server-side before calling Square.	[ ] Verify
Webhook verification	Always verify Square's HMAC signature before processing.	[ ] Verify
Error messages	Don't expose raw API errors to the frontend.	[ ] Verify
CORS	Restrict /api/checkout to your domain only if possible.	[ ] Verify
Rate limiting	Consider basic rate limiting on /api/checkout.	[ ] Verify

---

10. File Reference Summary

These are all the files that need to be created or modified:

File	Action	Purpose
api/checkout.js	CREATE	Backend endpoint — calls Square API, returns checkoutUrl

File	Action	Purpose
api/webhooks/square.js	CREATE	Webhook handler — verifies and processes Square events
js/cart.js	CREATE	Cart state management — add, remove, update, persist
js/checkout.js	CREATE	Checkout trigger — calls /api/checkout, redirects to Square
.env	CREATE	Local environment variables (do not commit)
.env.example	CREATE	Template showing required env vars (safe to commit)
All product pages	MODIFY	Replace Buy Now links with Add to Cart buttons
Site header/nav	MODIFY	Add cart icon with item count badge
/order-confirmed page	CREATE	Post-payment success page
package.json	MODIFY	Add squareup as a dependency

11. Environment Variable Template

Create this as .env.example in your repo root (safe to commit — no real values):

```
# Square API Credentials
# Get these from https://developer.squareup.com

# Use sandbox values during development:
SQUARE_ACCESS_TOKEN=
SQUARE_LOCATION_ID=
SQUARE_ENVIRONMENT=sandbox

# Webhook signature key (from Square Dashboard → Webhooks)
SQUARE_WEBHOOK_SIGNATURE_KEY=

# Set SQUARE_ENVIRONMENT=production for live deployment
```

12. Reference Links

Resource	URL
Square Developer Dashboard	https://developer.squareup.com
Square Checkout API Docs	https://developer.squareup.com/docs/checkout-api
Create Payment Link (API Ref)	https://developer.squareup.com/reference/square/checkout-api/create-payment-link
Square Node.js SDK	https://github.com/square/square-nodejs-sdk
Webhooks Guide	https://developer.squareup.com/docs/webhooks/overview

Resource	URL
Square Sandbox Test Cards	https://developer.squareup.com/docs/testing/test-values
Vercel Serverless Functions	https://vercel.com/docs/functions
Netlify Functions	https://docs.netlify.com/functions/overview/

End of Document

Squirrel Made Products · squirrelmadeproducts.com · 2026